

LLM Notes

2026 Spring

Day 9: CS336 Transformer LM (1)

Deep Learning

主讲: Aaron

Fudan University

今日摘要

2026.7.11 完成了 CS336 Assignment 1 中截至 Transformer LM 的编程作业。本文记录这一阶段的实现心得。并整理 Transformer 参数量、显存占用与 FLOPs 核算部分的非代码作业。

9.1 编程实现中的技巧与心得

9.1.1 einops 的使用

第一次比较正式地学习并使用了 `einops`。在需要频繁拆分、重排和合并张量维度时，它对于提高代码可读性非常有帮助。

`einops` 的一个重要特点，是能够将张量计算与形状说明结合起来。例如，使用 `rearrange` 拆分多头注意力时，可以直接在表达式中写出

$$(\text{batch}, \text{sequence}, d_{\text{model}}) \longrightarrow (\text{batch}, \text{head}, \text{sequence}, d_{\text{head}}),$$

从而使维度变化更加直观，也减少了连续使用 `view`、`reshape` 和 `transpose` 时可能产生的错误。

主要使用了以下两个函数：

- `rearrange`：完成张量维度的拆分、交换、合并与重新排列；
- `einsum`：使用带有维度名称的表达式描述矩阵乘法和张量收缩。

`einops reduce` 操作也是常用的。

9.1.2 对 `torch.nn.Module` 的进一步理解

过去使用过不少 PyTorch 函数，也曾在 MiniMind 项目中搭建过语言模型。不过，当时更多是在调用已经封装好的组件，对 PyTorch 如何管理一个模型的内部结构并没有形成特别清晰的认识。

CS336 要求尽量从底层手写各个组件。除了继承 `torch.nn.Module` 之外，许多常用模块都需要自行实现，例如 Linear、Embedding、RMSNorm、SwiGLU、RoPE 和多头自注意力。

- 使用 `nn.Parameter` 注册可训练参数；
- 将子模块保存为 `self.xxx`，使其被父模块递归管理；
- 使用 `register_buffer` 保存不参与训练、但需要随模型移动的张量；
- 使用 `nn.ModuleList` 注册并管理多个 Transformer Block；
- 使用 `state_dict` 保存、加载和检查模型中的参数；

一个 PyTorch 模型不仅是一组数学运算，同时也是一个具有参数注册、设备迁移、序列化和模块递归关系的完整软件结构。

9.1.3 对 Transformer 代码结构的理解

这部分作业中，我从底层的 Linear、Embedding、RMSNorm 和 SwiGLU 开始，逐步实现 RoPE、缩放点积注意力、多头因果自注意力、Transformer Block，最后将多个 Block 组合成完整的 Transformer LM。

过程中最大的困难并不只是理解单个公式，还包括正确管理：

- 张量的形状与维度顺序；
- 模块之间的层级关系；
- 权重名称与 adapter 接口；
- 多个 Transformer Block 的顺序执行。

9.1.3.1 Attention Mask

mask 应该作用在 softmax 之前的注意力分数矩阵上。

缩放点积注意力为

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V,$$

其中 M 表示 mask 对注意力分数施加的限制。

对于不允许关注的位置，可以将对应分数设置为

$$-\infty.$$

经过 softmax 后，这些位置的注意力权重就会变成 0。

无论是因果掩码，还是其他类型的 attention mask，本质上都作用在 query-key 两两配对形成的注意力分数矩阵上，而不是作用在最终的特征输出上。

9.1.3.2 RoPE

区分两个关键层次：

1. 沿特征维度，将相邻的两个标量组成一个二维向量；
2. 沿序列维度，根据 token position 为这些二维向量施加不同角度的旋转。

不同的特征维度对还会使用不同频率的旋转角度，从而让模型能够表示不同尺度的相对位置信息。在多头注意力中，RoPE 应作用于拆分多头之后的 Q 和 K ，而不作用于 V 。